Coastal Courier

VOL. 1, NO. 062

2026-03-30

COVER STORY

Three ideas about text messages

I'm aboard of a flight bringing me to San Francisco.

Antirez

Eventually I purchased the slowest internet connection of my life (well at least for a good reason), but for several hours I was without internet, as usually when I fly.

I don't mind staying disconnected for some time usually. It's a good time to focus, write some code, or a blog post like this one. However when I'm disconnected, what makes the most difference is not Facebook or Twitter or Github, but the lack of text messages.

At this point text messages are a fundamental thing in my life. They are also probably the main source of distraction. I use messages to talk with my family, even just to communicate between different floors. I use messages with friends to organize dinners and vacations. I even use messages with the plumber or the doctor.

Clearly messages are not going to go away. They need to get better, so I usually tend to thing at topics related to messaging applications. The following are three recurrent thoughts I've about this topic.

1. WhatsApp is not used in the US.

Do you know what's the social network that is reshaping the most the way we communicate in Europe? Is the WhatsApp application. Since it is the total standard and an incredible amount of the population has it, "WhatsApp Groups" is changing the way people communicate. There is a group for each school class, one for the children and one for the parents. One for families, another for groups of friends. One for the kindergarten of my daughter, where teachers post from time to time pictures of our children doing activities.

WhatsApp is also one of the main pains of modern life. All this groups continuously beep. However what they are doing for the society is incredible: they are truly making every human being interconnected. This magic is possible because here in Italy, and in most other EU countries, WhatsApp is *the standard* so everybody assumes you use it.

To me it is pretty incredible that this is not happening in the US, the place where most social applications are funded and created. Given that in the US there are both Android and iOS phones I wonder what's stopping this from happening. My guess is that it's just a matter of time and a unified messaging platform will happen there too.

* Why voice recognition is not used more.

For people that can write text fast into a real keyboard, the software keyboard of the phone is one of the most annoying things ever. Similarly, teenagers excluded, many other people have issues writing long text messages with a phone keyboard.

At the same time, the voice recognition software of Android, powered by the Google servers, reached a point where it rarely does errors at all, so why just a few people use it on a daily basis? My theory is that the user interface to activate and use the voice recognition feature is so bad to be the main barrier to make this feature a big hit.

First you have to find how to activate it, and usually is not a prominent button on the keyboard. Then there is this delay and it emits a beep and starts to listen, and it's not clear exactly how to stop it, especially if the environment is aloud. The whole thing looks like if you are suffering from the servers latency, but voice is voice. With delays and non clear experience you kill the big advantage of talking to your phone. This should be just a "push the button while you talk" thing. When you push the button, the system starts to record your voice immediately and connects asynchro-

nously to the servers. As text arrives back, it is shown to the user. When the user releases the button the voice thing is done. As simple as that. At the end, the words that were inserted in this way could be shown in some special way (like a different gray) in the text area, so that one can tap individual words and select alternatives if there are errors.

Please Google freaking do that. I wonder if it's any better on iOS, I don't use it for some time at this point.

* Auto transcription of text messages.

Since the phone keyboard is such a mess, people are using more and more voice messages, especially with WhatsApp. It is very convenient for people writing messages, since there is the "push and talk" experience that there is not in the speech to text feature. However it is not always very convenient for the people receiving the message, that may be in an environment where to listen to the conversation is not easy.

However there is something that could work in that regard, that is, the WhatsApp (or whatever) servers, should automatically translate the message to text, so that the user can both play the message or just look at the text, if it's clear enough. When there are doubts on given words, the text can be highlighted in some way to show the user that it's better to listen to the voice message since there are non clear words, however this could easily make 95% of messages promptly available to the user just reading.

This way the feature would be friction-less both ways, for the users writing (dictating actually) and for the user receiving the messages.

TLDR: Messaging is everywhere, make it better. Comments

{

FRONT PAGE
LAMBDA: The ultimate Excel worksheet function

Lambda the Ultimate

Post by Andy Gordon and Simon Peyton Jones on LAMBDA giving Excel users the ability to define functions.

Ever since it was released in the 1980s, Microsoft Excel has changed how people organize, analyze, and visualize their data, providing a basis for decision-making for the millions of people who use it each day. It's also the world's most widely used programming language. Excel formulas are written by an order of magnitude more users than all the C, C++, C#, Java, and Python programmers in the world combined. Despite its success, considered as a programming language Excel has fundamental weaknesses. Over the years, two particular shortcomings have stood out: (1) the Excel formula language really only supported scalar values—numbers, strings, and Booleans—and (2) it didn't let users define new functions.

Until now.

}

{

FEATURES

From where I left

Antirez

I'm not the kind of person that develops a strong attachment to their own work. When I decided to leave Redis, about 1620 days ago (4.44 years), I never looked at the source code, commit messages, or anything related to Redis again. From time to time, when I needed Redis, I just downloaded it and compiled it. I just typed "make" and I was very happy to see that, after many years, building Redis was still so simple.

My detachment was not the result of me hating my past work. While in the long run my creative work was less and less important and the "handling the project" activities became more and more substantial — a shift that many programmers are able to do, but that's not my bread and butter — well, I still enjoyed doing Redis stuff when I left. However, I don't share the vision that most people at my age (I'm 47 now) have: that they are still young. I wanted to do new stuff, especially writing. I wanted to stay more with my family and help my relatives. I definitely needed a break.

However, during the "writing years" (I'm still writing, by the way), I often returned to coding, as a way to take breaks from intense writing sessions (writing is the only mental activity I found to be a great deal more taxing than coding): I did a few embedded projects; played more with neural networks; built Telegram bots: a bit of everything. Hacking randomly was cool but, in the long run, my feeling was that I was lacking a real purpose, and every day I started to feel a bigger urgency to be part of the tech world again. At the same time, I saw the Redis community fragmenting, something that was a bit concerning to me, even as an outsider.

So I started to think that maybe, after all, I could have a role back in the Redis ecosystem. Perhaps I could be able to reshape the company's attitude towards the community. Maybe I could even help to take back the role of the Redis core as the primary focus of new developments. Basically I could be some kind of "evangelist" (I don't love the name of this role, but... well, you get it), that is, on one side, a bridge between the company and the community, but also somebody that could produce programming demos, invent and describe new patterns, write documentation, videos and blog posts about new and old stuff. And, what about the design of new stuff? I could learn from the work of people in the wild, from their difficulties, distill it, and report back design ideas, in order for Redis to evolve.

Time in NY

At some point my daughter, who is now 12, and is a crucial person in my life, enlightening my days with her intelligence, creativity and love, wanted to visit NYC for her birthday. We decided that yes, this was a good idea after all, we had a couple very difficult years recently, so, why not? My daughter is now more of a girl than a child. So, in NYC, I thought: maybe this is the right time, I can do a part time job. I met the new Redis CEO, Rowan Trollope, very recently, in a video call. I had the feeling that I could work with him to tune the future of the company's relationships with the community and the codebase direction. So I wrote him an email saying: do you think I could be back in some kind of capacity? Rowan showed interest in my proposal, and quickly we found some agreement.

About the license switch

People will ask questions about why I *actually* did this, whether there is some back story other than what I just wrote above, if there is some agreement involved, or a big amount of money; something odd or unclear. But sometimes things are very boring: 1. I contacted the company, not the reverse. 2. I'm not getting crazy money to re-enter, it's not about exploiting some situation — normal salary (but, disclaimer: yes, I have Redis stock options like I had before, no less, no more). 3. I don't have huge issues with Redis changing its license; Specifically I don't think the fracture with the community is *really* about this. But since people will ask me about that very important matter, it is better to tell you all the truth immediately.

The licensing dilemma

I wrote open source software for almost my whole life. Yet, as I'm an atheist and still I'm happy when I see other people believing in God, if this helps them to survive life's hardships, I also don't believe that open source is the only way to write software. When I started to develop Redis in the context of a company where I was one of the two founders and where we kept the software code closed (Redis was opened as it was considered not part of the core product). We didn't want our services to be copied by others, as simple as that. So I'm not an extremist in this regard – I'm an extremist only about software design.

Moreover, I don't believe that openness and licensing are only what the OSI tells us they are. I see licensing as a spectrum of things you can and can't do. At the same time, I'm truly concerned that big cloud providers have changed the incentives in the system software arena. Redis was not the only project to change license, it was actually the last one... of a big pile. And I have the feeling that in recent years many projects didn't even start because of a lack of a clear potential business model. So, the Redis license switch was not my decision and perhaps I would have chosen a different license? I'm not sure, it's too easy to relitigate right now, far from the scene for many years and without business pressures. But in general, I can understand the choice.

Moreover, if you read the new Redis license, sure, it's not BSD, but basically as long as you don't sell Redis as a service, you can use it in very similar ways and with similar freedoms as before (what I mean is that you can still modify Redis, redistribute it, use Redis commercially, in your for-profit company, for free, and so forth). You can *even* still sell Redis as a service if you want, as long as you release all the orchestration systems under the same license (something that nobody would likely do, but this shows the copyleft approach of the license). The license language is

```
}
{
```

Stack Overflow: The Architecture - 2016 Edition

Nick Craver

This is #1 in a very long series of posts on Stack Overflow's architecture. Welcome. Previous post (#0): Stack Overflow: A Technical Deconstruction Next post (#2): Stack Overflow: The Hardware - 2016 Edition

To get an idea of what all of this stuff "does," let me start off with an update on the average day at Stack Overflow. So you can compare to the previous numbers from November 2013 , here's a day of statistics from February 9th, 2016 with differences since November 12th, 2013:

209,420,973 (+61,336,090) HTTP requests to our load balancer

66,294,789 (+30,199,477) of those were page loads

1,240,266,346,053 (+406,273,363,426) bytes (1.24 TB) of HTTP traffic sent

569,449,470,023 (+282,874,825,991) bytes (569 GB) total received

3,084,303,599,266 (+1,958,311,041,954) bytes (3.08 TB) total sent

504,816,843 (+170,244,740) SQL Queries (from HTTP requests alone)

5,831,683,114 (+5,418,818,063) Redis hits

17,158,874 (not tracked in 2013) Elastic searches

3,661,134 (+57,716) Tag Engine requests

607,073,066 (+48,848,481) ms (168 hours) spent running SQL queries

10,396,073 (-88,950,843) ms (2.8 hours) spent on Redis hits

147,018,571 (+14,634,512) ms (40.8 hours) spent on Tag Engine requests

1,609,944,301 (-1,118,232,744) ms (447 hours) spent processing in ASP. Net

22.71 (-5.29) ms average (19.12 ms in ASP. Net) for 49,180,275 question page renders

11.80 (-53.2) ms average (8.81 ms in ASP. Net) for 6,370,076 home page renders

You may be wondering about the drastic ASP. Net reduction in processing time compared to 2013 (which was 757 hours) despite 61 million more requests a day. That's due to both a hardware upgrade in early 2015 as well as a lot of performance tuning inside the applications themselves. Please don't forget: performance is still a feature . If you're curious about more hardware specifics than I'm about to provide—fear not. The next post will be an appendix with detailed hardware specs for all of the servers that run the sites (I'll update this with a link when it's live).

So what's changed in the last 2 years? Besides replacing some servers and network gear, not much. Here's a top-level list of hardware that runs the sites today (noting what's different since 2013):

4 Microsoft SQL Servers (new hardware for 2 of them)

11 IIS Web Servers (new hardware)

2 Redis Servers (new hardware)

3 Tag Engine servers (new hardware for 2 of the 3)

3 Elasticsearch servers (same)

4 HAProxy Load Balancers (added 2 to support CloudFlare)

2 Networks (each a Nexus 5596 Core + 2232TM Fabric Extenders , upgraded to 10Gbps everywhere)

2 Fortinet 800C Firewalls (replaced Cisco 5525-X ASAs)

2 Cisco ASR-1001 Routers (replaced Cisco 3945 Routers)

2 Cisco ASR-1001-x Routers (new!)

What do we need to run Stack Overflow? That hasn't changed much since 2013 , but due to the optimizations and new hardware mentioned above, we're down to needing only 1 web server. We have unintentionally tested this, successfully, a few times. To be clear: I'm saying it works. I'm not saying it's a good idea. It's fun though, every time.

Now that we have some baseline numbers for an idea of scale, let's see how we make those fancy web pages. Since few systems exist in complete isolation (and ours is no exception), architecture decisions often make far less sense without a bigger picture of how those pieces fit into the whole. That's the goal here, to cover the big picture. Many subsequent posts will do deep dives into specific areas. This will be a logistical overview with hardware highlights only; the next post will have the hardware details.

For those of you here to see what the hardware looks like these days, here are a few pictures I took of rack A (it has a matching sister rack B) during our February 2015 upgrade :

style="text-align: center;">

...and if you're into that kind of thing, here's the entire 256 image album from that week (you're damn right that number's intentional). Now, let's dig into layout. Here's a logical overview of the major systems in play:

id="ground-rules">Ground Rules

Here are some rules that apply globally so I don't have to repeat them with every setup:

```
}
{
```

Side projects

Antirez

Today Redis is six years old. This is an incredible accomplishment for me, because in the past I switched to the next thing much faster. There are things that lasted six years in my past, but not like Redis, where after so much time, I still focus most of my everyday energies into.

How did I stopped doing new things to focus into an unique effort, drastically monopolizing my professional life? It was a too big sacrifice to do, for an human being with a limited life span. Fortunately I simply never did this, I never stopped doing new things.

If I look back at those 6 years, it was an endless stream of side projects, sometimes related to Redis, sometimes not.

1) Load81, children programming environment.

2) Dump1090, software defined radio ADS-B decoder.

3) A Javascript ray tracer.

4) lua-cmsgpack, C implementation of msgpack for Lua.

5) linenoise line editing library. Used in Redis, but well, was not our top priority.

6) lamernews, Redis-based HN clone.

7) Gitan, a small Git web interface.

8) shapeme, images evolver using simulated annealing.

9) Disque, a distributed queue (work in progress right now).

And there are much more throw-away projects not listed here.

The interesting thing is that many of the projects listed above are not random hacking efforts that had as an unique goal to make me happy. A few found their way into other people's code.

Because of the side projects, I was able to do different things when I was stressed and impoverished from doing again and again the same thing. I could later refocus on Redis, and find again the right motivations to have fun with it, because small projects are cool, but to work for years at a single project can provide more value for others in the long run.

So currently I'm using something like 20% of my time to hack on Disque, a distributed message queue. So only 80% is left for Redis development, right? Wrong. The deal is between 80% of focus on Redis and 20% on something else, or 0% of focus on Redis in the long term, because in order to have a long term engagement, you need a long term alternative to explore new things.

Side projects are the projects making your bigger projects possible. Moreover they are often the start of new interesting projects. Redis itself was a side project of LLOOGG. Sometimes you stop working at your main project because of side projects, but when this happens it is not because your side project captured your focus, it is because you managed to find a better use for your time, since the side project is more important, interesting, and compelling than the main project.

Redis is six years old today, but is aging well: it continues to capture the attention of more developers, and it continues to improve in order to provide a bit more value to users every week. However for me, more users, more pull requests, and more pressure, does not mean to change my setup. What Redis is today is the sum of the work we put into it, and the endurance in the course of six years. To continue along the same path, I'll make sure to have a few side projects for the next years.

UPDATE: Damian Janowski provided an incredible present for the Redis community today, the renewed Redis.io web site is online now! http://redis.io. Thanks Damian!

HN comments here: https://news.ycombinator.com/item?id=9112250 Comments

```
}
{
```

What is performance?

Antirez

The title of this blog post is an apparently trivial to answer question, however it is worth to consider a bit better what performance really means: it is easy to get confused between scalability and performance, and to decompose performance, in the specific case of database systems, in its different main components, may not be trivial. In this short blog post I'll try to write down my current idea of what performance is in the context of database systems.

A good starting point is probably the first slide I use lately in my talks about Redis. This first slide is indeed about performance, and says that performance is mainly three different things.

1) Latency: the amount of time I need to get the reply for a query.

2) Operations per unit of time per core: how many queries (operations) the system is able to reply per second, in a given reference computational unit?

3) Quality of operations: how much work those operations are able to accomplish?

—

This is probably the simplest component of performance. In many applications it is desirable that the time needed to get a reply from the system is small. However while the average time is important, another concern is the predictability of the latency figure, and how much difference there is between the average case and the worst case. When used well, in-memory systems are able to provide very good latency characteristics, and are also able to provide a consistent latency over time.

Operations per second per core

—

The second component I'm enumerating is what makes the difference between raw performance and scalability. We are interested in the amount of work the system is able to do, in a given unit of time, for a given reference computational unit. Linearly scalable systems can reach a big number of operations per second by using a number of nodes, however this means they are scalable, and not necessarily performant.

Operations per second per core is also usually bound to the amount of queries you can perform per watt, so to the energy efficiency of the system.

Quality of operations

—

The last point, while probably not as stressed among developers as throughput and latency, is really important in certain kind of systems, especially in-memory systems.

A system that is able to perform 100 operations per second, but with operations of "poor quality" (for example just GET and SET in Redis terms) has a lower performance compared to a system that is also able to perform an INCR operation with the same latency and OPS characteristics.

For instance, if the problem at hand is to increment counters, the former system will require two operations to increment a counter (we are not considering race conditions in this context), while the system providing INCR is able to use a single operation. As a result it is actually able to provide twice the performance of the former system.

As you can see the quality of operations is not an absolute meter, but depends on the kind of problem to solve. The same two systems if we want to cache HTML fragments are equivalent since the INCR operation would be useless.

The quality of operations is particularly important in in-memory systems, since usually the computation itself is negligible compared to the time needed to receive, dispatch the command, and create a reply, so systems like Redis with a rich set of operations are able to provide better performance in many contexts almost for free, just allowing the user to do more with a single operation. The "do more" part can actually mean a lot of things: either provide a reply to a more complex question, like for example the ZRANK command of Redis, or simply being able to provide a more *selective* reply, like HMGET command that is able to provide information just for a subset of the fields composing an Hash value, reducing the amount of bandwidth required between the server and its clients.

In general quality of operations don't only affect performances because they give less or more value to the operations per second the system is able to perform: operations quality also directly affect latency, since more complex operations are able to avoid back and forth data transfer between clients and servers required to mount multiple simpler operations into a more complex computation.

—

I hope that this short exploration of what performance is uncovered some of the complexities involved in the process of evaluating the capabilities of a database system from this specific point of view. There is a lot more to say about it, but I found that the above three components of the performance are among the most interesting and important when evaluating a system and when there is to understand how to evolve an existing system to improve its performance characteristics.

Thanks to Yiftach Shoolman for feedbacks about this topic.
Comments

}

standard-article(title: [Redis as AP system, reloaded], author: [Antirez], source-name: [Antirez], images: (), paragraphs: ([So finally something really good happened from the Redis criticism thread.], [At the end of the work day I was reading about Redis as AP and merge operations on Twitter. At the same time I was having a private email exchange with Alexis Richardson (from RabbitMQ, and, my boss). Alexis at some point proposed that perhaps a way to improve safety was to asynchronously ACK the client about what commands actually were not received so that the client could retry. This seemed a lot of efforts in the client side, but somewhat totally opened my view on the matter.], [So the idea is, we can't go for synchronous replication, but indeed we get ACKs from the replicas, asynchronous ACKS, specifically.], [What about retaining all the writes not acknowledged into a buffer, and "re-play" them to the current master when the partition heals?], [The window we need to require to take the log is very small if the ACKs are frequent enough (currently the frequency is 1 per second, but this could be more easily).], [If we give up Availability after N times the window we can say, ok, no more room, we now start to reply with errors to queries.], [The HUGE difference with this approach is that this works regardless of the size of values. There are also semantical differences since the stream of operations is preserved instead of the value itself, so there is more context. Think for example about INCR.], [Of course this would not work for anything, but one could mark in the command table what command to reply and what to discard. SADD is an example of perfect command since the order of operations does not matter. DEL is likely something to avoid replying. And so forth. In turn if we reply against the wrong (stale) master, it will accumulate the commands and so forth. Details may vary, but this is the first thing that really makes a difference.], [Probably many of you that are into eventually consistent databases know about the log VS merge strategies already, but I had to re-invent the wheel as I was not aware. This is the kind of feedback

standard-article(title: [Getting Started with ES6 Modules], author: [Robin Ward (eviltrout)], source-name: [Robin Ward (eviltrout)], images: (), paragraphs: ([Javascript is a fantastic example of how something, despite having visible warts and very poor design , can dominate the tech landscape. Nobody uses Javascript because it's a beautiful language; they use it because it's ubiquitous. Its warts are now well understood and most have workarounds.], [An amazing omission in Javascript's design is the lack of a built-in module system. As more projects used Javascript and shared more code, the need for a robust module system became necessary. Two contenders sprung up, Asynchronous Module Definition (AMD) and CommonJS (CJS). The former is much more popular with browser applications and the latter is much more popular with server applications written in node.js .],), insert-map: (:), word-count: 112, edited-for-length: false, debug-mode: false,)

brief-group(([

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Recalling our series on Conway's Game of Life, here is an implementation of Life within Life. Unfortunately, it does not "prove" what I hoped it might, so unless a reader has a suggestion on what this demonstration proves, it doesn't belong in the proof gallery. But it sure is impressive.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): I've been spending a little less time on my blog recently then I'd like to, but for good reason: I've been attending two weeks of research conferences, I'm getting ready for a summer internship in cybersecurity, and I've finally chosen an advisor. Visions, STOC, and CCC I've been taking a break from the Midwest for the last two weeks to attend some of this year's seminal computer science theory conferences. The first (which is already over) was the Simon's Institute Symposium on the Visions of the Theory of Computing.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Steven Clontz informed me of an effort he's involved in called code4math. It's described as a professional organization for the advancement of mathematical research through building non-research software infrastructure. By that he means, for example, writing software packages like Macaulay2 or databases of mathematical objects that other researchers can use to do their research. Clontz recently gave a talk on the topic, with ample discussion of the evaluation material they can provide to justify the academic value of this sort of work.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): This post assumes working knowledge of elementary number theory. Luckily for the non-mathematicians, we cover all required knowledge and notation in our number theory primer. So Three Thousand Years of Number Theory Wasn't Pointless It's often tough to come up with concrete applications of pure mathematics. In fact, before computers came along mathematics was used mostly for navigation, astronomy, and war. In the real world it almost always coincided with the physical sciences.

], [

I expected in the Redis thread that I did not received.], [Another cool thing about this approach is that it's pretty opt-in, it can be just a state in the connection. Send a command and the connection is of "safe" type, so all the commands sent will be retained and replayed if not acknowledged, and so forth.], [This is not going to be in the first version of Redis Cluster as I'm more happy to ship ASAP the current design, but it is a solid incremental idea that could be applied later, so a little actual result into the evolution of the design. Comments],), insert-map: (:), word-count: 464, edited-for-length: false, debug-mode: false,)

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Problem: Show that all horses are of the same color. "Solution": We will show, by induction, that for any set of $ n$ horses, every horse in that set has the same color. Suppose $ n=1$, this is obviously true. Now suppose for all sets of $ n$ horses, every horse in the set has the same color. Consider any set $ H$ of $ n+1$ horses. We may pick a horse at random, $ h_1 \in H$, and remove it from the set, getting a set of $ n$ horses.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Cellular Automata There is a long history of mathematical models for computation. One very important one is the Turing Machine, which is the foundation of our implementations of actual computers today. On the other end of the spectrum, one of the simpler models of computation (often simply called a system) is a cellular automaton. Surprisingly enough, there are deep connections between the two. But before we get ahead of ourselves, let's see what these automata can do.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Dangling Nodes and Non-Uniqueness Recall where we left off last time. Given a web $ W$ with no dangling nodes, the link matrix for $ W$ has 1 as an eigenvalue, and if the corresponding eigenspace has dimension 1, then any associated eigenvector gives a ranking of the pages in $ W$ which is consistent with our goals. The first problem is that if there is a dangling node, our link matrix has a column of all zeros, and is no longer column-stochastic.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Decidability Versus Efficiency In the early days of computing theory, the important questions were primarily about decidability. What sorts of problems are beyond the power of a Turing machine to solve? As we saw in our last primer on Turing machines, the halting problem is such an example: it can never be solved a finite amount of time by a Turing machine. However, more recently (in the past half-century) the focus of computing theory has shifted away from possibility in favor of determining feasibility.

], [

Freek Van der Herten, Freek Van der Herten: Every page in Oh Dear now works on mobile. Not just slapped-on media

queries, but reworked layouts: a floating action button for navigation, dedicated mobile card views for monitor lists, scrollable tables with fade hints, and bigger touch targets throughout. Over 160 Blade templates were touched.

Read more

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Previously in this series we've seen the definition of a category and a bunch of examples, basic properties of morphisms, and a first look at how to represent categories as types in ML. In this post we'll expand these ideas and introduce the notion of a universal property. We'll see examples from mathematics and write some programs which simultaneously prove certain objects have universal properties and construct the morphisms involved.

], [

Jeremy Kun (Math ∩ Programming), Jeremy Kun (Math ∩ Programming): Not Just Time, But Space Too! So far on this blog we've introduced models for computation, focused on Turing machines and given a short overview of the two most fundamental classes of problems: P and NP. While the most significant open question in the theory of computation is still whether P = NP, it turns out that there are hundreds (almost 500, in fact!) other "classes" of problems whose relationships are more or less unknown.

],))

standard-article(title: [Memory lane for Frontier users], author: [Scripting News], source-name: [Scripting News], images: (), paragraphs: ([I had to find out which domains being served by a problem server were still mapping to its domain. This server had been running for six years, and I was pretty sure some of the apps had moved.], [So I wrote a script in Frontier, it was the best tool available to me, and got my answer in 20 minutes, code written from scratch.], [The script visited each subfolder, the filename is the domain of the folder, finds out which server it's supposed to be running on, based on a DNS lookup, and adds a line to a list.], [Here's a screen shot of the domains folder.], [Here's the script as a screen shot and GitHub doc .], [This is just a way to preserve a little of the Frontier culture. Hard to explain in words. Easier to show as screen shots.],), insert-map: (:), word-count: 142, edited-for-length: false, debug-mode: false,)

standard-article(title: [Google announces Logica: organizing your data queries, making them universally reusable and fun], author: [Lambda the Ultimate], source-name: [Lambda the Ultimate], images: (), paragraphs: ([You can read more about it at the Google Open Source blog post, Logica: organizing your data queries, making them universally reusable and fun .], [They advocate for datalog-like language they developed internally at Google.], [The reason?], [Good programming is about creating small, understandable, reusable pieces of logic that can be tested, given names, and organized into packages which can later be used to construct more useful pieces of logic. SQL resists this workflow. Although you can encapsulate certain repeated computations into views and functions, the syntax and support for these can vary among implementations, the notions of packages and imports are generally nonexistent, and higher-level constructions (e.g. passing a function to a function) are impossible.],), insert-map: (:), word-count: 116, edited-for-length: false, debug-mode: false,)

standard-article(title: [], author: [Scripting News], source-name: [Scripting News], images: (), paragraphs: ([Podcast : Jen and I often exchange voicemails. Yesterday I sent one about how she, who is not a programmer, should try creating software with Claude or ChatGPT. I think the hardest

standard-article(title: [Enemy of the State], author: [Robin Ward (eviltrout)], source-name: [Robin Ward (eviltrout)], images: (), paragraphs: ([I learned very quickly while working on a large open source project is that it is important to make my code hard to break. The primary line of defense

part is figuring out how to get it to give you a file that you can run from your own desktop. But I explained that in the voicemail. Midway through I realized this a podcast, and checked with her if it would be okay and she was very emphatic that I should. You see NJ aside to being one of my best friends for life, is also a Natural Born Blogger or a person with maximum audacity. Her first instinct like mine is to share it and shut up. So that's what I'm doing. As usual I asked Claude to write the show notes. Hope you like it and thanks for listening!],), insert-map: (:), word-count: 144, edited-for-length: false, debug-mode: false,)

for this is a comprehensive test suite, but I think it's also very important to create functions that are easy to use and difficult to damage.], [I find I even code this way on personal projects that will never be released. Even if you never work on a team with other developers, there is a good chance you will forget a lot of implementation details of the code that you aren't actively working on. You need to protect your code from yourself!],), insert-map: (:), word-count: 111, edited-for-length: false, debug-mode: false,)

{

Scripting News

A bit of history. Read this post from 20 years ago by Phil Jones. That's what I was trying to do back then, just as Twitter came online. I didn't know it then but was the moment when the web stopped growing. When the VCs took over, and monetized the hell out of it. What we got in the end was Trump and Musk. We would have been smart, as a civilization, to hedge against the monopolies. If we get another chance what are we going to do with it? Will we work together this time? It's worth one more shot. My comments on the Jones piece in 2006 and 2026 .

}

Coastal Courier

Vol. 1, No. 062 · 2026-03-30

Curated and composed automatically.